

Cụm 4 - Quality Attributes

Mục lục

4.1 Tổng quan Cụm 4: Quality Attributes	3
Vì sao có cụm này	3
Cụm 4 sẽ dạy gì	3
Bài 4.2: Functional vs Non-functional Requirements	3
Bài 4.3: Identifying Architecture Characteristics	3
Bài 4.4: Component-Based Thinking	3
Quality Attributes là gì?	3
Bảng QA phổ biến	4
Operational (runtime)	4
Structural (development-time)	4
Cross-cutting	4
“You can’t pick all of them”	4
Performance □ Security	5
Consistency □ Availability (CAP theorem)	5
Scalability □ Simplicity	5
Time-to-market □ Maintainability	5
Cost □ Performance	5
Privacy □ Observability	5
Architect’s responsibility	5
Cách đọc cụm này	5
4.2 Functional vs Non-functional Requirements	6
Câu hỏi mở đầu	6
Định nghĩa formal	6
Functional Requirement (FR)	6
Non-functional Requirement (NFR)	6
Architecture Characteristic (Mark Richards)	7
Implicit vs Explicit Characteristics	7
Explicit	7
Implicit	7
“Architecture is the X-ilities”	7
Ví dụ thực hành: từ FR □ NFR □ Architecture	8
Use case: Hệ booking phim cinema	8
NFR khó hơn FR	8
Khó measure	8
Khó test	8
Khó negotiate với business	9

Conflict nhau	9
Sai lầm thường gặp	9
Sai lầm 1: Bỏ qua NFR trong design phase	9
Sai lầm 2: NFR không measurable	9
Sai lầm 3: Treat tất cả NFR như nhau	9
Sai lầm 4: NFR không liên kết với business value	9
Tóm tắt	9
4.3 Xác định Architecture Characteristics	10
Vì sao “không quá 7”?	10
Pipeline identify	10
Bước 1: Đọc requirement document	10
Bước 2: Stakeholder interview	10
Bước 3: Implicit characteristics	11
Bước 4: Priority với business	11
Bước 5: Operationalize	11
Hierarchy của Characteristics	12
Operational (Runtime characteristics)	12
Structural (Development characteristics)	12
Cross-cutting	12
CQA Worksheet (Quality Attribute Workshop simplified)	13
Sai lầm thường gặp	13
Sai lầm 1: “Mọi thứ đều quan trọng”	13
Sai lầm 2: QA không measurable	13
Sai lầm 3: Implicit không document	13
Sai lầm 4: QA conflict không resolve	13
Sai lầm 5: Không revisit	13
Tóm tắt	14
4.4 Component-Based Thinking	15
Component là gì?	15
Component vs Module — phân biệt	15
Identify Component	15
1. Entity-based (theo domain entity)	15
2. Workflow-based (theo use case)	16
Combined approach	16
Component Cohesion Principles (Robert C. Martin)	16
REP — Reuse/Release Equivalence Principle	16
CCP — Common Closure Principle	16
CRP — Common Reuse Principle	17
Tension giữa 3 principles	17
Component Coupling Principles	17
ADP — Acyclic Dependencies Principle	18
SDP — Stable Dependencies Principle	18
SAP — Stable Abstractions Principle	18
Main Sequence	18
Sai lầm thường gặp	19
Sai lầm 1: Component quá nhỏ	19
Sai lầm 2: Component quá lớn	19

Sai lầm 3: Cyclic dependencies tolerated	19
Sai lầm 4: Component theo layer (horizontal slicing)	19
Tóm tắt	19
Tổng kết Cụm 4	19

4.1 Tổng quan Cụm 4: Quality Attributes

Tóm tắt một dòng: Bạn không thể tối ưu mọi thứ cùng lúc — phải chọn 3-7 quality attributes ưu tiên cho hệ thống của mình. Cụm này dạy cách identify, prioritize, và translate chúng thành quyết định kiến trúc cụ thể.

Vì sao có cụm này

Cụm 3 dạy “trade-off là essence của architecture”. Nhưng trade-off *giữa cái gì*? Câu trả lời: giữa các **quality attributes** (còn gọi là **architecture characteristics**, **non-functional requirements**, hay **-ilities**).

Không hiểu QA thì:

- Trade-off analysis trở thành argument cảm tính (“tôi thấy nó tốt hơn”).
- Quyết định kiến trúc không có “lý do” rõ ràng, không thuyết phục được stakeholder.
- Khi requirement thay đổi, không biết quyết định nào cần re-evaluate.

Cụm này fix cả 3 vấn đề.

Cụm 4 sẽ dạy gì

Bốn bài:

Bài 4.2: Functional vs Non-functional Requirements

Phân biệt requirement chức năng (cái hệ phải làm) với non-functional (cách hệ phải làm). Vì sao QA quan trọng hơn function trong nhiều case. Định nghĩa Architecture Characteristic theo Mark Richards (3 tiêu chí: non-domain consideration, structural impact, critical to success).

Bài 4.3: Identifying Architecture Characteristics

Cách extract QA từ requirement document, từ stakeholder interview, từ business context. Top-N rule: chọn không quá 7 QA priority. Hierarchy: operational, structural, cross-cutting. Cách dùng ATAM/CQA worksheet để negotiate priorities với business.

Bài 4.4: Component-Based Thinking

Cách map QA xuống component design. Component vs Module vs Class. Identify component boundary từ entity vs workflow approach. Component cohesion principles (REP, CCP, CRP) và component coupling principles (acyclic, stable, abstract).

Quality Attributes là gì?

Definition: **non-domain consideration** ảnh hưởng đến *cách hệ thống behave* hoặc *được build*. Khác biệt với functional requirement:

Aspect	Functional	Non-functional (QA)
Câu hỏi	Hệ làm gì?	Hệ làm thế nào?

Aspect	Functional	Non-functional (QA)
Test	Pass/fail (acceptance test)	Đo đếm (benchmark, monitoring)
Example	“User đặt được order”	“Order processing p99 < 200ms”
Owner	Product / Business	Architecture / Engineering

Bảng QA phổ biến

Theo Bass-Clements-Kazman, *Software Architecture in Practice* 4th ed:

Operational (runtime)

QA	Định nghĩa
Performance	Response time, throughput
Scalability	Khả năng tăng load mà vẫn duy trì performance
Availability	Uptime, fault tolerance
Security	Confidentiality, integrity, access control
Reliability	Đúng output, không corrupt data
Recoverability	Time-to-recover sau failure

Structural (development-time)

QA	Định nghĩa
Maintainability	Effort để sửa bug / thêm feature
Modularity	Mức tách bạch giữa các phần
Testability	Effort để test code
Deployability	Effort + time để deploy
Reusability	Code/component dùng lại được ở project khác

Cross-cutting

QA	Định nghĩa
Cost (CAPEX/OPEX)	Tiền infrastructure + dev
Time-to-market	Thời gian từ ý tưởng đến production
Auditability	Khả năng trace ai làm gì khi nào
Observability	Hiểu state hệ qua logs, metrics, traces
Privacy	Tuân thủ GDPR, CCPA, dữ liệu cá nhân
Accessibility	Hỗ trợ disabilities
Internationalization (i18n)	Đa ngôn ngữ

Tổng cộng ~20-30 QA phổ biến. Một hệ không thể tối ưu cả 30 — phải chọn 5-7 ưu tiên.

“You can’t pick all of them”

Định luật bất khả: **Tối ưu một QA thường hy sinh QA khác**. Vài cặp trade-off kinh điển:

Performance □ Security

Mã hoá dữ liệu tăng security nhưng giảm performance. Cache giảm latency nhưng tăng attack surface.

Consistency □ Availability (CAP theorem)

Hệ phân tán không thể đạt cả Consistency + Availability khi có Network partition. Phải chọn. Bài 6.4 (Event-Driven) sẽ đi sâu.

Scalability □ Simplicity

Microservices scale tốt nhưng phức tạp hơn monolith. Kafka throughput cao nhưng phức tạp hơn HTTP.

Time-to-market □ Maintainability

Hack quick & dirty ship nhanh nhưng tech debt cao. Code đúng SOLID + test full ship chậm nhưng maintain lâu dài.

Cost □ Performance

GPU nhanh hơn CPU nhưng đắt. Replica cluster tăng availability nhưng x3 cost.

Privacy □ Observability

Log mọi user action giúp debug nhưng leak PII. Anonymize logs lose context.

Architect's responsibility

Mỗi quyết định kiến trúc nên:

1. **Tham chiếu QA ưu tiên** đã agree với business.
2. **Show trade-off** ngắn gọn (Bài 3.3).
3. **Document** trong ADR (Bài 7.4 sẽ dạy).

Nếu architect ra quyết định không tham chiếu QA □ quyết định cảm tính, dễ bị challenge và overrule sau đó.

Cách đọc cụm này

Tuần tự 4.2 □ 4.3 □ 4.4. Mỗi bài 3500-4500 chữ.

Mục tiêu thực hành sau cụm:

- List 5-7 QA ưu tiên cho một hệ thực mà bạn đang làm.
- Đo được (hoặc đề xuất cách đo) cho mỗi QA.
- Vẽ component diagram với boundary thể hiện QA.

Vào [Bài 4.2: Functional vs Non-functional](#) để bắt đầu.

4.2 Functional vs Non-functional Requirements

Tóm tắt một dòng: Functional requirement nói “hệ phải làm X”; Quality attribute nói “hệ phải làm X tốt cỡ nào”. Architecture được quyết định bởi QA chứ không phải bởi functional requirement — vì cùng functionality có thể được implement bằng vô số architecture khác nhau.

Câu hỏi mở đầu

Hai team build cùng một e-commerce site. Functional requirement giống hệt: catalog, cart, checkout, payment.

Team A architect:

- Monolithic Rails app.
- PostgreSQL.
- Deploy 1 server EC2 t3.medium.
- Cost: \$30/tháng.

Team B architect:

- 12 microservices (Catalog, Cart, Inventory, Payment, ...) trên Kubernetes.
- PostgreSQL + Redis + Elasticsearch + Kafka.
- Multi-region active-active deployment.
- Cost: \$15,000/tháng.

Cả hai đều “đáp ứng functional requirement” — user có thể browse, add to cart, checkout. Nhưng architecture khác nhau 500 lần về cost. Tại sao?

Vì *non-functional requirement* khác nhau:

- Team A: ~100 user, không cần multi-region, downtime 1 giờ/tháng OK.
- Team B: 10M user, 99.99% uptime, p99 latency < 200ms, scale to 10x traffic in 5 phút.

Architecture decision được driven bởi NFR, không phải bởi functional requirement. Đây là một trong những insight quan trọng nhất của Software Architecture.

Định nghĩa formal

Functional Requirement (FR)

“What the system must do.” — Hành vi observable mà hệ phải có.

Ví dụ: - “User có thể đăng ký account bằng email + password.” - “Admin có thể export CSV danh sách user.” - “Hệ tự động gửi email khi order được ship.”

FR pass/fail qua acceptance test: hành vi xảy ra hoặc không.

Non-functional Requirement (NFR)

“How well the system must do it.” — Constraint trên cách hệ behave.

Ví dụ: - “API response p99 < 200ms cho 95% endpoints.” - “Uptime ≥ 99.9% (3 phút downtime/tháng).” - “Hỗ trợ 10,000 concurrent users.” - “Code 80%+ unit test coverage.”

NFR đo qua benchmark, monitoring, code metric. Có thể “đạt mức nào đó” — không pass/fail binary.

Architecture Characteristic (Mark Richards)

Mark Richards trong *Fundamentals of Software Architecture* dùng từ “architecture characteristic” thay vì NFR. Định nghĩa:

“An architecture characteristic meets three criteria:”

1. Specifies a **non-domain design consideration**.
2. **Influences some structural aspect** of the design.
3. **Critical or important** to application success.

Phân tích 3 tiêu chí:

1. Non-domain: không thuộc về business logic. Vd “Order phải có tax” là domain rule. “Order processing latency < 200ms” là non-domain — không nói về domain order, nói về *cách* processing.

2. Structural impact: yêu cầu thay đổi structure của hệ. Vd “100 concurrent user” có thể serve bởi single server. “10M concurrent user” require horizontal scaling, load balancer, có lẽ CDN — structure khác hoàn toàn.

3. Critical: nếu không đạt, hệ fail. Vd: “Pinterest cần search latency < 100ms” — nếu 5 giây thì user rời ngay. “Internal admin tool 2-second latency OK” — không critical.

Mọi NFR architectural đều thoả 3 tiêu chí. Một số NFR không thoả tiêu chí 2 (vd “code 80% test coverage” không structural) □ không architectural.

Implicit vs Explicit Characteristics

Explicit

Stakeholder nói rõ trong requirement doc. Vd: “API p99 < 200ms” — viết trong SLA.

Implicit

Stakeholder không nói nhưng *vẫn expect*. Vd:

- User expect “hệ không bị hack” — security implicit (trừ khi product là hack tool, ha ha).
- User expect “data không bị mất khi crash” — reliability implicit.
- Developer expect “build < 5 phút” — maintainability implicit.

Architect job: identify implicit characteristics. Stakeholder không có ngôn ngữ để articulate chúng — architect đào ra.

Pattern: với mỗi feature, hỏi:

- “Nếu feature này down 1 ngày, user phản ứng sao?”
- “Nếu data sai 1%, ảnh hưởng gì?”
- “Nếu attacker exploit endpoint này, hậu quả?”
- “Nếu hệ scale lên 10x, gì breaks first?”

Câu trả lời tiết lộ implicit characteristics.

“Architecture is the X-ilities”

Trong industry, NFR thường gọi là “-ilities” vì nhiều cái kết thúc bằng -ility:

- Reliability
- Scalability

- Maintainability
- Testability
- Deployability
- Observability
- Auditability
- Availability

Quote nổi tiếng (không rõ tác giả): “Architecture is the X-ilities. Get them right and your system works. Get them wrong and your system collapses.”

Reason: -ilities là cái khó retro-fit. Function add thêm vào hệ cũ được. Performance/scalability/security bake vào ngay từ đầu mới đảm bảo.

Ví dụ thực hành: từ FR → NFR → Architecture

Use case: Hệ booking phim cinema

Functional requirements: - User search phim theo ngày, rạp, thể loại. - User chọn ghế, payment. - User nhận email confirmation + QR code. - Admin manage suất chiếu, giá vé.

Non-functional requirements (cần đào):

- *Performance*: search response < 500ms (user wait không bị nản).
- *Concurrency*: thứ bảy 7pm peak 50,000 concurrent user nationwide.
- *Consistency*: 2 user không được book cùng ghế (strong consistency for seat selection).
- *Availability*: ≥ 99.5% uptime, đặc biệt cao ngày cuối tuần.
- *Scalability*: peak khoảng 10x normal, scale up trong < 5 phút.
- *Security*: payment information theo PCI DSS.
- *Auditability*: log mọi transaction cho refund support.

Architecture implications:

- *Concurrency 50k + strong consistency seat*: cần seat reservation pattern (Redis SETNX hoặc database row lock). Không cache seat availability.
- *Peak 10x*: auto-scaling group, scale-out architecture (stateless services).
- *PCI DSS*: payment processing isolated service, không touch app-level code. Stripe/local gateway.
- *Audit*: append-only event log (Event Sourcing optional, audit log mandatory).

Quyết định kiến trúc *suy ra* từ NFR. Khác NFR → khác architecture.

NFR khó hơn FR

NFR thường:

Khó measure

“Maintainable” cụ thể là gì? Cyclomatic complexity? Lines of code? Time to onboard new dev?

Solution: operationalize NFR bằng metric cụ thể. “Maintainable” = “new dev onboard < 2 tuần đến first PR” + “average bugfix time < 4 giờ” + “cyclomatic complexity < 15 per method”.

Khó test

Performance test cần load generator. Scalability cần production-like environment. Security cần penetration test. Cost cao.

Khó negotiate với business

Business muốn “everything fast and cheap”. Architect job: present trade-off, force prioritization.

Tool: ATAM, CQA (Continuous Quality Assurance) workshop — sẽ học ở 4.3.

Conflict nhau

Đã nói ở 4.1: cặp trade-off. Không pick all.

Sai lầm thường gặp**Sai lầm 1: Bỏ qua NFR trong design phase**

Team build feature theo functional spec, không hỏi NFR. Production ship rồi mới đau: too slow, không scale, security holes.

Fix: NFR phải elicited *trước* khi viết code. Workshop 1-2 giờ với business để extract NFR.

Sai lầm 2: NFR không measurable

“Hệ phải fast.” □ useless. Phải định lượng: “p99 < 200ms”.

Sai lầm 3: Treat tất cả NFR như nhau

Stakeholder say “tất cả NFR đều critical”. Sai. Phải force priority. 5-7 NFR top.

Sai lầm 4: NFR không liên kết với business value

NFR phải justify bằng business impact. “p99 < 200ms vì study cho thấy mỗi 100ms tăng latency làm conversion giảm 1%”. Không justify được □ cắt khỏi requirement.

Tóm tắt

- **FR**: hệ làm gì. **NFR**: hệ làm tốt cỡ nào.
- **Architecture decision driven bởi NFR**, không phải FR.
- **Architecture characteristic** (Richards): non-domain + structural impact + critical.
- **Implicit characteristics**: architect đào ra từ stakeholder.
- **NFR khó**: measure, test, negotiate, conflict.

Bài tiếp: [Identifying Architecture Characteristics](#) — kỹ thuật extract NFR từ requirement.

4.3 Xác định Architecture Characteristics

Tóm tắt một dòng: Đừng pick “all the -ilities”. Chọn 3-7 QA top driven bởi business priority, document chúng có thể đo được, dùng workshop với stakeholder để negotiate khi xung đột.

Vì sao “không quá 7”?

Cognitive limit. Người ta nhớ và optimize được khoảng 5-9 thứ cùng lúc (Miller’s 7 ± 2 rule). Architect đặt 15 QA “quan trọng” □ 15 QA đều không thực sự được tối ưu.

Quy tắc: **Top 5-7 QA**. Còn lại là implicit (default expectations như “không crash”, “không leak password”) nhưng không driven decision.

Tỷ lệ thực hành:

- 3 QA: lý tưởng cho startup MVP. Vd: “time-to-market, simplicity, cost”.
- 5 QA: ổn cho hệ trung. Vd: “performance, scalability, availability, maintainability, security”.
- 7 QA: max. Vd: thêm “auditability, observability”.
- 10+ QA: bạn đang nói dối rằng mọi thứ đều ưu tiên. Force prioritize.

Pipeline identify

Bước 1: Đọc requirement document

Highlight mọi từ tiềm năng là NFR. Vài keyword:

- “fast”, “responsive”, “real-time” □ performance
- “scale”, “grow”, “millions” □ scalability
- “uptime”, “always available”, “24/7” □ availability
- “secure”, “encrypted”, “compliant” □ security
- “audit”, “trace”, “log” □ auditability
- “easy to deploy”, “frequent release” □ deployability
- “low cost”, “budget” □ cost

Lưu ý: business owner thường không dùng từ kỹ thuật. “Hệ phải always work” có thể nghĩa availability hoặc reliability. Phải hỏi để clarify.

Bước 2: Stakeholder interview

Schedule meeting 1-2h với:

- Product Owner / PM (business priorities).
- Engineering Manager (team capability).
- Customer Success (user pain points hiện tại).
- Optional: end user (vd: focus group cho consumer product).

Câu hỏi mẫu:

- “Trong 3 năm tới, hệ này có khả năng phát triển ra sao? 10x user? 100x?” □ scalability priority.
- “Nếu hệ down 1 giờ giữa giờ làm, ai bị ảnh hưởng và thiệt hại bao nhiêu?” □ availability priority.
- “Tốc độ release feature hiện tại là tuần? tháng? Có cần nhanh hơn không?” □ time-to-market + deployability.
- “Có khả năng audit / compliance requirement (GDPR, SOC2, PCI)?” □ compliance/auditability.
- “Hệ này lưu data nhạy cảm? Loại nào?” □ security/privacy.

Bước 3: Implicit characteristics

Như đã nói ở Bài 4.2, ngoài explicit, còn implicit. Default implicit cho most systems:

- Reliability: không data corruption.
- Security: basic auth, no XSS/SQLi.
- Maintainability: ai cũng có thể onboard.
- Observability: logs + basic metrics.

Implicit không cần “top 7” — nhưng cần *baseline*. Architect đảm bảo baseline được respected.

Bước 4: Priority với business

Workshop format:

1. List 10-15 candidate QA (mix explicit + implicit).
2. Business stakeholder rank: must-have / should-have / nice-to-have.
3. Push back: nếu mọi cái “must-have”, force chọn top 5.
4. Output: top 5-7 QA + acceptable range cho mỗi cái.

Ví dụ output:

QA	Priority	Target
Performance	Must	p99 < 200ms cho 95% endpoint
Availability	Must	99.9% uptime
Security	Must	Compliance OWASP Top 10
Scalability	Should	Scale 10x trong 1 năm
Maintainability	Should	Onboard new dev < 2 tuần
Cost	Should	< \$5k/tháng infra cho 50k users

Document này là “north star” cho mọi architecture decision.

Bước 5: Operationalize

Translate QA thành metric đo được.

QA	Cách đo
Performance	p50, p95, p99 latency từ APM (Datadog, New Relic)
Availability	Uptime % từ monitoring (Pingdom, internal)
Scalability	Max concurrent users từ load test
Maintainability	Mean time to PR-merged, cyclomatic complexity
Deployability	Time from commit to production
Security	Pen-test results, dependency vulnerability count
Cost	Monthly cloud bill

Mỗi QA nên có:

- Metric đo (number).
- Cách đo (tool).

- Threshold pass/fail (vd: p99 < 200ms = pass; > 200ms = alert).
- Frequency review (vd: weekly architecture review).

Không operationalize được □ QA không “real”, chỉ là khẩu hiệu.

Hierarchy của Characteristics

Có thể group QA theo 3 layer:

Operational (Runtime characteristics)

QA về *cách hệ chạy* trong production:

- Performance
- Scalability
- Availability
- Reliability
- Security
- Recoverability

Đo bằng monitoring tool.

Structural (Development characteristics)

QA về *cách hệ được build*:

- Maintainability
- Testability
- Deployability
- Modularity
- Reusability

Đo bằng code metric + dev workflow metric.

Cross-cutting

QA xuyên qua mọi layer:

- Cost
- Time-to-market
- Auditability
- Observability
- Compliance (GDPR, HIPAA, ...)
- Accessibility
- i18n

Đo theo context.

Insight: thường top 5-7 QA distributed across 3 layer. Vd:

- 2 operational (performance, availability).
- 2 structural (maintainability, deployability).
- 1-2 cross-cutting (cost, compliance).

Distribution này khoẻ. Tất cả ở 1 layer là warning (bias).

CQA Worksheet (Quality Attribute Workshop simplified)

Format compact để run workshop 60-90 phút:

QA	Scenario (concrete)	Source	Response	Measure
Performance	User search product on mobile during peak	User	Result returned	p99 < 300ms
Scalability	Black Friday traffic 10x normal	System	All requests served	< 5% error rate
Availability	DB primary fails	System	Failover to replica	< 30s outage
Maintainability	Junior dev fix critical bug	Dev	PR merged	< 4 hours
Security	Attacker tries SQL injection	Attacker	Request blocked	0 successful attacks/year

Mỗi row là một “QA scenario”. Scenario cụ thể để test hơn QA abstract. Output workshop = bảng scenarios với measure rõ ràng.

Reference: ATAM (Architecture Tradeoff Analysis Method) của SEI Carnegie Mellon — full version dùng cho hệ critical. Quality Attribute Workshop simplified dùng cho hệ thường.

Sai lầm thường gặp

Sai lầm 1: “Mọi thứ đều quan trọng”

Phổ biến nhất. Stakeholder nói tất cả must-have. Architect không push back → end up với 15 QA = không QA nào được tối ưu thật.

Fix: force ranking. “Nếu chỉ chọn 3, bạn chọn cái nào?”.

Sai lầm 2: QA không measurable

“Hệ phải user-friendly.” → không đo được. Refine thành: “task completion rate > 85% trong usability test với 10 user.”

Sai lầm 3: Implicit không document

Implicit characteristics được architect assume nhưng không communicate. Sau 1 năm team thay đổi, no one biết. Document chúng vào “implicit baseline” section của architecture doc.

Sai lầm 4: QA conflict không resolve

List “performance + security + simplicity” mà không acknowledge chúng có thể conflict. Khi conflict xảy ra trong development, không có guidance để pick.

Fix: trong workshop, present cặp QA có thể conflict và force prioritization. Vd: “Khi performance vs security xung đột, mình chọn cái nào?”.

Sai lầm 5: Không revisit

QA được set 1 lần ở project kickoff rồi quên. Business priority có thể thay đổi (vd: COVID làm e-commerce ưu tiên availability hơn cost).

Fix: revisit QA mỗi quarter hoặc khi có major business change.

Tóm tắt

- **Top 5-7 QA** — không nhiều hơn.
- **Pipeline**: read requirement □ interview □ identify implicit □ priority workshop □ operationalize.
- **Hierarchy**: operational + structural + cross-cutting.
- **CQA worksheet**: scenario-based, measurable.
- **Sai lầm**: “mọi thứ quan trọng”, QA không measurable, implicit không doc, conflict không resolve, không revisit.

Bài tiếp (cuối Cụm 4): [Component-Based Thinking](#) — map QA xuống component.

4.4 Component-Based Thinking

Tóm tắt một dòng: Component là building block của architecture (lớn hơn class, có boundary rõ, có thể deploy/test độc lập). Identify component bằng entity-based hoặc workflow-based; tổ chức theo 3 cohesion + 3 coupling principles của Robert C. Martin.

Component là gì?

Robert C. Martin định nghĩa trong *Clean Architecture*:

“Components are the units of deployment. They are the smallest entities that can be deployed as part of a system.”

Examples cụ thể:

- Trong Java: JAR file.
- Trong .NET: DLL file.
- Trong Ruby: Gem.
- Trong JavaScript: npm package.
- Trong Python: wheel / module.
- Trong microservices: deployed service.

Key property: **independently deployable**. Component có boundary rõ — vào ra qua interface công khai. Bên trong là chi tiết.

Component khác:

- **Class**: nhỏ hơn. Một component thường chứa nhiều class.
- **Module** (theo nghĩa code organization): tương đương component ở một số ngữ cảnh, nhưng “module” thường mức code, “component” thường mức deployment.

Component vs Module — phân biệt

Nuance:

- **Module**: package code organization. Mục đích: navigation + namespace. Vd: Python package `myapp.orders`.
- **Component**: deployable unit. Mục đích: deployment + versioning. Vd: `orders-service.jar`.

Trong monolith, có nhiều module nhưng 1 component (1 deployable). Trong microservices, mỗi service là 1 component.

Cụm 5 (Architecture Styles) sẽ trở lại: mỗi style có cách khác nhau để compose component.

Identify Component

Hai phương pháp phổ biến:

1. Entity-based (theo domain entity)

Mỗi entity quan trọng □ component own entity đó.

Vd e-commerce:

- CustomerService component own Customer entity.
- OrderService component own Order entity.
- ProductService component own Product entity.
- PaymentService component own Payment entity.

Lợi ích: bounded context rõ. Mỗi component có data ownership.

Hạn chế: nếu một workflow dùng nhiều entity (vd: “place order” dùng Customer + Order + Product + Inventory + Payment), workflow phải cross 5 component $\square$ coupling cao.

2. Workflow-based (theo use case)

Mỗi workflow quan trọng $\square$ component own workflow đó.

Vd:

- OrderPlacementService cover toàn bộ flow “place order” (orchestrate Customer + Inventory + Payment internally).
- CustomerOnboardingService cover flow “đăng ký + verify email + setup profile”.

Lợi ích: workflow stay coherent, ít cross-component call.

Hạn chế: data ownership phức tạp (Customer entity bị edit bởi cả OrderService và OnboardingService).

Combined approach

Thực tế thường mix:

- **Aggregate entities** thành Bounded Context (DDD): OrderManagement context own Order, OrderItem, OrderStatus entities.
- **Workflow** dùng cross-context: CheckoutFlow orchestrate OrderManagement + Payment + Shipping.

Mỗi approach có pros/cons. Quyết định theo: số workflow cross-entity, frequency thay đổi, team structure.

Component Cohesion Principles (Robert C. Martin)

Ba nguyên tắc cho việc *cái gì nên ở cùng component*:

REP — Reuse/Release Equivalence Principle

“The granule of reuse is the granule of release.”

Diễn đạt: bất cứ thứ gì bạn release together (cùng version) phải reusable together. Người dùng component không thể chọn release một phần của component — họ get tất cả hoặc không gì.

Hệ quả: classes trong component nên có *common purpose*. Vd: react-router package chỉ chứa routing-related code, không lẫn react-i18n.

CCP — Common Closure Principle

“Gather into components those classes that change for the same reasons and at the same times.”

Diễn đạt: class thay đổi cùng nhau nên ở cùng component. Khi requirement thay đổi, ideally chỉ 1 component bị touch.

Hệ quả: chia component theo *axis of change*. Vd: nếu UI hay thay đổi separately từ business logic, tách thành 2 component.

Note: CCP scale up SRP từ class lên component.

CRP — Common Reuse Principle

“Don’t force users of a component to depend on things they don’t need.”

Diễn đạt: nếu user dùng class X từ component C, user shouldn’t be forced to depend trên class Y trong C mà user không dùng.

Hệ quả: classes có usage pattern khác nhau nên ở component khác nhau. Vd: Database và Cache thường được dùng cùng nhau □ cùng component OK. Nhưng EmailSender không được dùng cùng □ tách component.

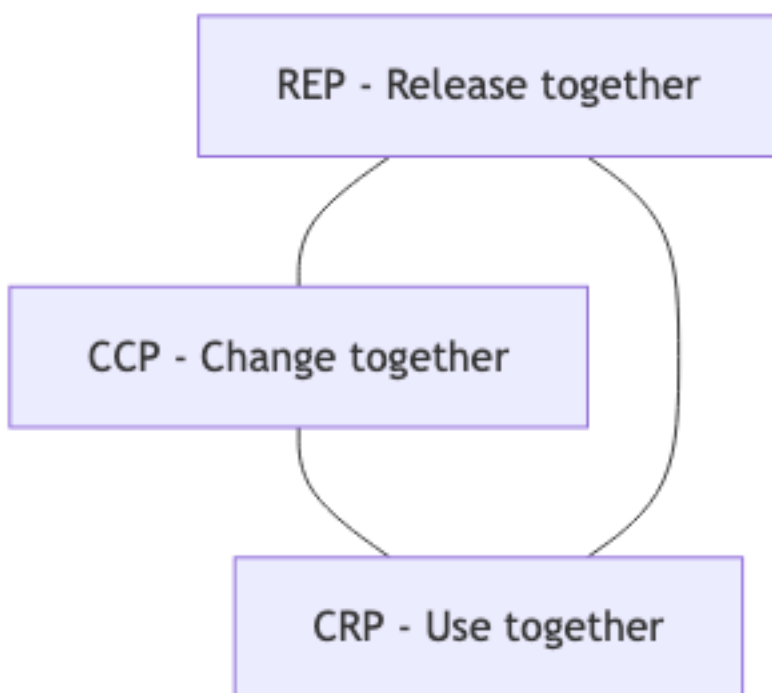
Note: CRP scale up ISP lên component.

Tension giữa 3 principles

Trade-off:

- REP + CCP push toward larger components (more in 1 component).
- CRP push toward smaller components (less in 1 component).

Triangle:



Hình 1: Sơ đồ 9

3 cạnh không thể đều đạt 100% — phải balance. Nguyên tắc:

- Early stage: prioritize CCP + REP (tooling/team focus on flexibility).
- Mature stage: shift to CRP (tooling/team focus on stability).

Component Coupling Principles

Ba nguyên tắc cho *cách component depend lẫn nhau*:

ADP — Acyclic Dependencies Principle

“Allow no cycles in the component dependency graph.”

Nếu $A \rightarrow B \rightarrow C \rightarrow A$ cycle $\rightarrow$ cả 3 phải release cùng nhau, mất ý nghĩa tách component.

Detect: tool như `jdepend` (Java), `madge` (JS), `pydeps` (Python) visualize dependency graph + báo cycles.

Fix: - **Move dependency**: tách class gây cycle ra component khác. - **Dependency Inversion**: introduce abstraction để break cycle (DIP từ Bài 2.7).

SDP — Stable Dependencies Principle

“Depend in the direction of stability.”

Component thay đổi nhiều (volatile) nên depend vào component thay đổi ít (stable). Không ngược lại.

Lý do: nếu stable component depend volatile, mỗi lần volatile thay đổi $\rightarrow$ stable phải re-release $\rightarrow$ stable hết stable.

Đo stability: ratio $I = C_e / (C_a + C_e)$ với:

- C_a (afferent coupling): số component depend vào component này.
- C_e (efferent coupling): số component component này depend vào.
- $I = 0$: maximally stable (chỉ bị depend, không depend ai).
- $I = 1$: maximally unstable.

SAP — Stable Abstractions Principle

“A component should be as abstract as it is stable.”

Component stable nên abstract (interface), component volatile có thể concrete.

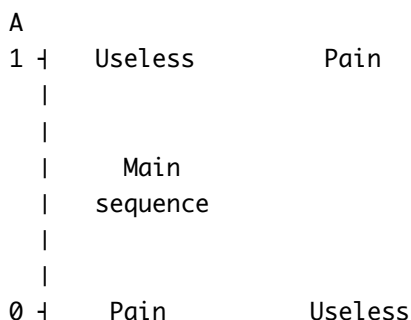
Lý do: stable component không nên chứa code chi tiết (vì khó thay đổi). Nó nên chứa abstraction, để concrete implementation ở volatile components.

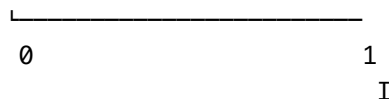
Đo abstractness: ratio $A = \text{abstract_classes} / \text{total_classes}$.

Main Sequence

SDP + SAP cho relationship I vs A:

- Pain zone: $I = 0, A = 0$ — stable concrete (rigid, khó thay đổi).
- Useless zone: $I = 1, A = 1$ — unstable abstract (no purpose).
- Main sequence: $I + A = 1$ — ideal line.





Component nên gần Main Sequence. Far from it = warning sign.

Sai lầm thường gặp

Sai lầm 1: Component quá nhỏ

Tách thành 100 component cho project medium. Build time chậm. Dependency hell. Coordination cost cao.

Quy tắc: component count khoảng = team count * 2-5. Vd: team 5 → 10-25 components.

Sai lầm 2: Component quá lớn

Tách quá lười. Component to → khó test, khó deploy parts riêng, vi phạm CCP.

Quy tắc: component khoảng 5-50 class (cho mediocre complexity).

Sai lầm 3: Cyclic dependencies tolerated

“Tạm thời” cycle tồn tại nhiều năm. Refactor khó hơn theo thời gian. Mỗi build slow vì cycle force rebuild widely.

Quy tắc: zero tolerance cho cycle ở component level. Detect + fix immediately.

Sai lầm 4: Component theo layer (horizontal slicing)

Đã nói ở 3.4. Component-by-layer = distributed monolith. Component nên by domain (vertical).

Tóm tắt

- **Component** = independently deployable unit, lớn hơn class.
- **Identify**: entity-based hoặc workflow-based (thường mix).
- **Cohesion principles**: REP (release together), CCP (change together), CRP (use together) — trade-off.
- **Coupling principles**: ADP (no cycle), SDP (depend toward stable), SAP (stable = abstract).
- **Main Sequence**: balance I + A = 1.

Tổng kết Cụm 4

Hết Cụm 4. Tóm tắt:

Bài	Insight chính
4.2 FR vs NFR	Architecture driven bởi NFR, không FR
4.3 Identifying	Top 5-7 QA, measurable, prioritize qua workshop
4.4 Component thinking	REP/CCP/CRP cohesion + ADP/SDP/SAP coupling, Main Sequence

Cụm 5-6 (Architecture Styles) sẽ áp dụng tất cả: mỗi style là một cách compose component để tối ưu một bộ QA cụ thể.

Bài tiếp: [Tổng quan Cụm 5 - Fundamental Architecture Styles](#).